Pandas Practice Questions

This notebook covers essential pandas operations including data loading, filtering, grouping, handling missing values, and data manipulation.

In [2]:
```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Create sample data for demonstration
np.random.seed(42)
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank', 'Grace', 'Henry',
    'Age': [25, 30, 35, 28, 32, 29, 27, 31, 26, 33],
    'Salary': [50000, 60000, 70000, 55000, 65000, np.nan, 58000, 72000, 52000, 6800
    'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'HR', 'Finance', 'IT', 'HR',
    'Experience': [2, 5, 8, 3, 6, 4, 2, 7, 1, 9],
    'Performance_Score': [85, 92, 88, 90, 87, 89, np.nan, 94, 86, 91]
}

df = pd.DataFrame(data)
print("Sample dataset created successfully!")
```

```
Sample dataset created successfully!
```

1. Load Dataset and Display First Rows

Load a dataset and display the first few rows along with basic information about the dataset.

In [3]:
```
# Display basic information about the dataset
print("Dataset Shape:", df.shape)
print("\nColumn Names:")
print(df.columns.tolist())
print("\nData Types:")
print(df.dtypes)
print("\nFirst 5 rows:")
print(df.head())
print("\nDataset Info:")
print(df.info())
print("\nBasic Statistics:")
print(df.describe())
```

```
Dataset Shape: (10, 6)

Column Names:
['Name', 'Age', 'Salary', 'Department', 'Experience', 'Performance_Score']

Data Types:
Name                 object
Age                   int64
Salary              float64
Department           object
Experience            int64
Performance_Score    float64
dtype: object

First 5 rows:
      Name  Age   Salary Department  Experience  Performance_Score
0    Alice   25  50000.0         IT           2               85.0
1      Bob   30  60000.0         HR           5               92.0
2  Charlie   35  70000.0         IT           8               88.0
3    Diana   28  55000.0    Finance           3               90.0
4      Eve   32  65000.0         IT           6               87.0

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Name               10 non-null     object
 1   Age                10 non-null     int64
 2   Salary             9 non-null      float64
 3   Department         10 non-null     object
 4   Experience         10 non-null     int64
 5   Performance_Score  9 non-null      float64
dtypes: float64(2), int64(2), object(2)
memory usage: 608.0+ bytes
None

Basic Statistics:
             Age        Salary  Experience  Performance_Score
count  10.000000      9.000000   10.000000           9.000000
mean   29.600000  61111.111111    4.700000          89.111111
std     3.204164   8022.537698    2.750757           2.934469
min    25.000000  50000.000000    1.000000          85.000000
25%    27.250000  55000.000000    2.250000          87.000000
50%    29.500000  60000.000000    4.500000          89.000000
75%    31.750000  68000.000000    6.750000          91.000000
max    35.000000  72000.000000    9.000000          94.000000
```

2. Filter Rows and Show Specific Columns

Filter the dataset based on certain conditions and display specific columns.

In [4]:
```
# Filter employees with age > 30
print("Employees with age > 30:")
filtered_age = df[df['Age'] > 30]
print(filtered_age[['Name', 'Age', 'Department']])
print()

# Filter IT department employees
print("IT Department employees:")
it_employees = df[df['Department'] == 'IT']
print(it_employees[['Name', 'Salary', 'Experience']])
print()

# Multiple conditions: IT employees with experience > 5
print("IT employees with experience > 5 years:")
experienced_it = df[(df['Department'] == 'IT') & (df['Experience'] > 5)]
print(experienced_it[['Name', 'Age', 'Experience', 'Salary']])
print()

# Filter by salary range (excluding NaN values)
print("Employees with salary between 55000 and 70000:")
salary_range = df[(df['Salary'] >= 55000) & (df['Salary'] <= 70000)]
print(salary_range[['Name', 'Salary', 'Department']])
```

```
Employees with age > 30:
      Name  Age Department
2  Charlie   35         IT
4      Eve   32         IT
7    Henry   31         IT
9     Jack   33    Finance

IT Department employees:
      Name   Salary  Experience
0    Alice  50000.0           2
2  Charlie  70000.0           8
4      Eve  65000.0           6
7    Henry  72000.0           7

IT employees with experience > 5 years:
      Name  Age  Experience   Salary
2  Charlie   35           8  70000.0
4      Eve   32           6  65000.0
7    Henry   31           7  72000.0

Employees with salary between 55000 and 70000:
      Name   Salary Department
1      Bob  60000.0         HR
2  Charlie  70000.0         IT
3    Diana  55000.0    Finance
4      Eve  65000.0         IT
6    Grace  58000.0    Finance
9     Jack  68000.0    Finance
```

3. Group by Target and Calculate Means

Group the data by department and calculate mean values for numerical columns.

In [5]:
```
# Group by Department and calculate means
print("Mean values by Department:")
dept_means = df.groupby('Department').mean(numeric_only=True)
print(dept_means)
print()

# More detailed groupby analysis
print("Detailed statistics by Department:")
dept_stats = df.groupby('Department').agg({
    'Age': ['mean', 'min', 'max'],
    'Salary': ['mean', 'median', 'count'],
    'Experience': ['mean', 'std'],
    'Performance_Score': ['mean', 'count']
})
print(dept_stats)
print()

# Group by multiple columns
print("Average salary by Department and Experience level:")
df['Experience_Level'] = df['Experience'].apply(lambda x: 'Junior' if x <= 3 else '
multi_group = df.groupby(['Department', 'Experience_Level'])['Salary'].mean()
print(multi_group)
```

```
Mean values by Department:
                 Age        Salary  Experience  Performance_Score
Department
Finance     29.333333  60333.333333    4.666667               90.5
HR          28.333333  56000.000000    3.333333               89.0
IT          30.750000  64250.000000    5.750000               88.5

Detailed statistics by Department:
                 Age                Salary                Experience  \
                mean min max          mean   median count       mean
Department
Finance     29.333333  27  33  60333.333333  58000.0     3   4.666667
HR          28.333333  26  30  56000.000000  56000.0     2   3.333333
IT          30.750000  25  35  64250.000000  67500.0     4   5.750000

                   Performance_Score
                 std              mean count
Department
Finance     3.785939              90.5     2
HR          2.081666              89.0     3
IT          2.629956              88.5     4

Average salary by Department and Experience level:
Department  Experience_Level
Finance     Junior              56500.0
            Senior              68000.0
HR          Junior              52000.0
            Senior              60000.0
IT          Junior              50000.0
            Senior              69000.0
Name: Salary, dtype: float64
```

4. Handle Missing Values

Identify and handle missing values in the dataset using various strategies.

In [6]:
```python
# Check for missing values
print("Missing values in each column:")
print(df.isnull().sum())
print()

print("Percentage of missing values:")
missing_percent = (df.isnull().sum() / len(df)) * 100
print(missing_percent)
print()

# Display rows with missing values
print("Rows with missing values:")
rows_with_missing = df[df.isnull().any(axis=1)]
print(rows_with_missing)
print()

# Create a copy for handling missing values
df_cleaned = df.copy()
```

```
# Strategy 1: Fill missing salary with mean
mean_salary = df_cleaned['Salary'].mean()
df_cleaned['Salary'].fillna(mean_salary, inplace=True)

# Strategy 2: Fill missing performance score with median
median_performance = df_cleaned['Performance_Score'].median()
df_cleaned['Performance_Score'].fillna(median_performance, inplace=True)

print("After handling missing values:")
print(df_cleaned.isnull().sum())
print("\nCleaned dataset:")
print(df_cleaned)
```

```
Missing values in each column:
Name                0
Age                 0
Salary              1
Department          0
Experience          0
Performance_Score   1
Experience_Level    0
dtype: int64

Percentage of missing values:
Name                 0.0
Age                  0.0
Salary              10.0
Department           0.0
Experience           0.0
Performance_Score   10.0
Experience_Level     0.0
dtype: float64

Rows with missing values:
    Name  Age   Salary Department  Experience  Performance_Score  \
5  Frank   29      NaN         HR           4               89.0
6  Grace   27  58000.0    Finance           2                NaN

  Experience_Level
5           Senior
6           Junior

After handling missing values:
Name                0
Age                 0
Salary              0
Department          0
Experience          0
Performance_Score   0
Experience_Level    0
dtype: int64

Cleaned dataset:
      Name  Age        Salary Department  Experience  Performance_Score  \
0    Alice   25  50000.000000         IT           2               85.0
1      Bob   30  60000.000000         HR           5               92.0
2  Charlie   35  70000.000000         IT           8               88.0
3    Diana   28  55000.000000    Finance           3               90.0
4      Eve   32  65000.000000         IT           6               87.0
5    Frank   29  61111.111111         HR           4               89.0
6    Grace   27  58000.000000    Finance           2               89.0
7    Henry   31  72000.000000         IT           7               94.0
8      Ivy   26  52000.000000         HR           1               86.0
9     Jack   33  68000.000000    Finance           9               91.0

  Experience_Level
0           Junior
1           Senior
2           Senior
```

```
3           Junior
4           Senior
5           Senior
6           Junior
7           Senior
8           Junior
9           Senior
```

```
C:\Users\dell\AppData\Local\Temp\ipykernel_45160\3347296340.py:22: FutureWarning: A
value is trying to be set on a copy of a DataFrame or Series through chained assignm
ent using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because
the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform
the operation inplace on the original object.


  df_cleaned['Salary'].fillna(mean_salary, inplace=True)
C:\Users\dell\AppData\Local\Temp\ipykernel_45160\3347296340.py:26: FutureWarning: A
value is trying to be set on a copy of a DataFrame or Series through chained assignm
ent using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because
the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform
the operation inplace on the original object.


  df_cleaned['Performance_Score'].fillna(median_performance, inplace=True)
```

5. Create New Column and Find Top 3 Values

Create new calculated columns and identify top performers in the dataset.

In [7]:
```python
# Create new columns
df_final = df_cleaned.copy()

# 1. Salary per year of experience
df_final['Salary_per_Experience'] = df_final['Salary'] / df_final['Experience']

# 2. Performance category
df_final['Performance_Category'] = df_final['Performance_Score'].apply(
    lambda x: 'Excellent' if x >= 90 else 'Good' if x >= 85 else 'Average'
)

# 3. Total score (weighted combination)
df_final['Total_Score'] = (df_final['Performance_Score'] * 0.6) + (df_final['Experi

print("Dataset with new columns:")
print(df_final[['Name', 'Salary_per_Experience', 'Performance_Category', 'Total_Sco
print()
```

```
# Find top 3 employees by different criteria
print("Top 3 employees by Salary:")
top_salary = df_final.nlargest(3, 'Salary')[['Name', 'Salary', 'Department']]
print(top_salary)
print()

print("Top 3 employees by Performance Score:")
top_performance = df_final.nlargest(3, 'Performance_Score')[['Name', 'Performance_S
print(top_performance)
print()

print("Top 3 employees by Total Score:")
top_total = df_final.nlargest(3, 'Total_Score')[['Name', 'Total_Score', 'Performanc
print(top_total)
print()

print("Top 3 employees by Salary per Experience:")
top_efficiency = df_final.nlargest(3, 'Salary_per_Experience')[['Name', 'Salary_per
print(top_efficiency)
```

```
Dataset with new columns:
      Name  Salary_per_Experience Performance_Category  Total_Score
0    Alice           25000.000000                 Good         73.5
1      Bob           12000.000000            Excellent         95.2
2  Charlie            8750.000000                 Good        110.3
3    Diana           18333.333333            Excellent         83.0
4      Eve           10833.333333                 Good         98.2

Top 3 employees by Salary:
      Name   Salary Department
7    Henry  72000.0         IT
2  Charlie  70000.0         IT
9     Jack  68000.0    Finance

Top 3 employees by Performance Score:
    Name  Performance_Score Department
7  Henry               94.0         IT
1    Bob               92.0         HR
9   Jack               91.0    Finance

Top 3 employees by Total Score:
      Name  Total_Score Performance_Category
9     Jack        116.1            Excellent
2  Charlie        110.3                 Good
7    Henry        106.9            Excellent

Top 3 employees by Salary per Experience:
    Name  Salary_per_Experience  Experience
8    Ivy                52000.0           1
6  Grace                29000.0           2
0  Alice                25000.0           2
```